

# Example

- **Asset:** QQQ (Invesco QQQ Trust)
- **Frequency:** Daily stock prices
- **Date Range:** March 28, 2020 – March 27, 2022
- **Fields Included:** Close price, Minimum price (Low), Maximum price (High)
- **Dim:** 502 (days) by 3 (prices)

Min-max scaler:

QQQ from 2020-03-28 to 2022-03-27:

|                   |        |        |        |        |        |     |      |          |          |          |
|-------------------|--------|--------|--------|--------|--------|-----|------|----------|----------|----------|
| Daily stock price | x0     | x1     | x2     | x3     | x4     | ... | x499 | x500     | x501     | x502     |
|                   | min x0 | min x1 | min x2 | min x3 | min x4 | ... |      | min x500 | min x501 | min x502 |
|                   | max x0 | max x1 | max x2 | max x3 | max x4 | ... |      | max x500 | max x501 | max x502 |

Min and Max

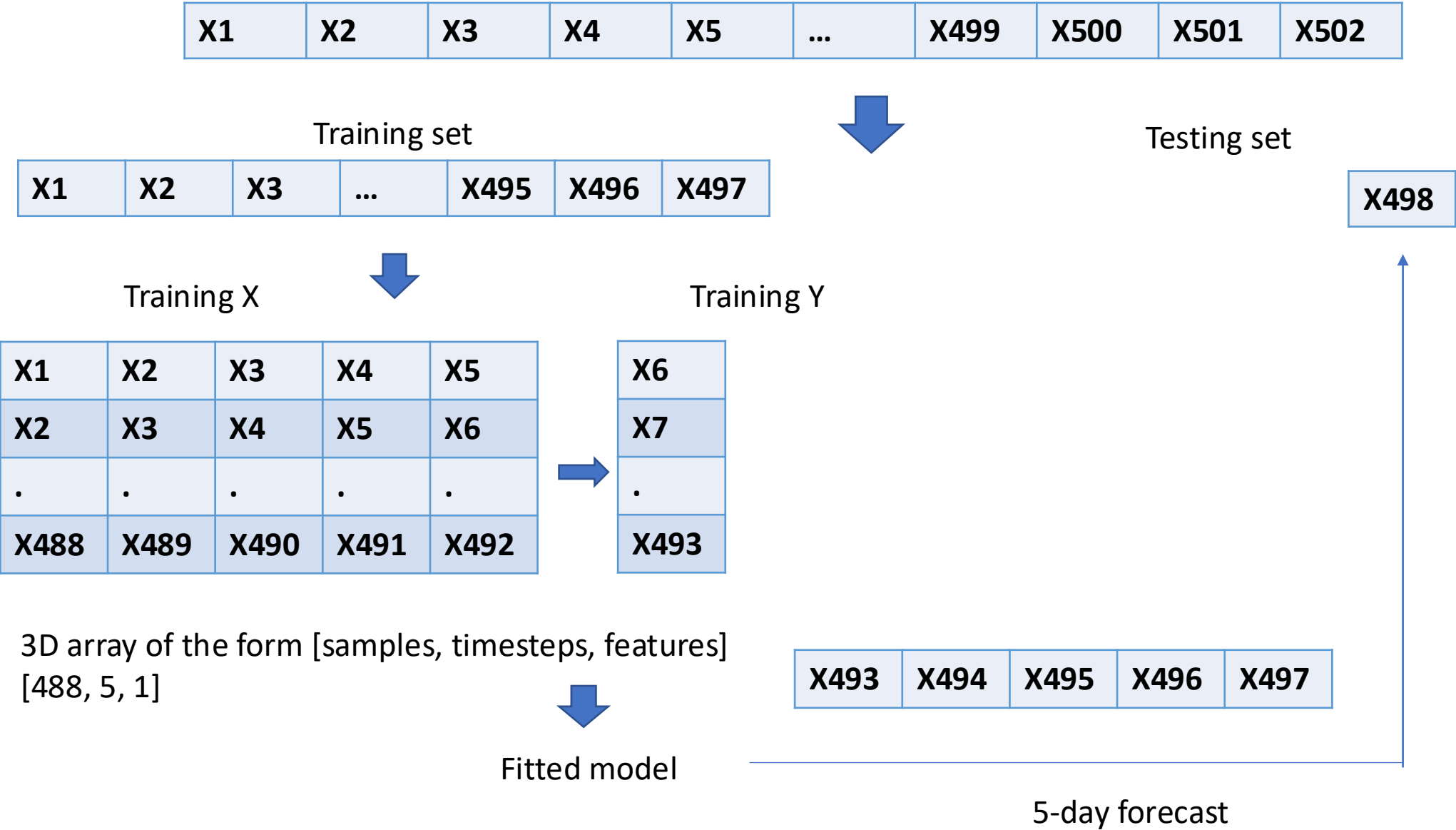
Scaled X1 = (x1 – min x0)/(max x0 – min x0)



$$\text{Scaled}_t = \frac{x_t - \min(x_1, \dots, x_{t-1})}{\max(x_1, \dots, x_{t-1}) - \min(x_1, \dots, x_{t-1})}$$

|                    |    |    |    |    |    |     |      |      |      |      |
|--------------------|----|----|----|----|----|-----|------|------|------|------|
| Scaled stock price | x1 | x2 | x3 | x4 | x5 | ... | x499 | x500 | x501 | x502 |
|--------------------|----|----|----|----|----|-----|------|------|------|------|

QQQ from 2020-03-28 to 2022-03-27:



## older style / not recommended in keras3

```
model <- keras_model_sequential()  
model %>%  
  layer_lstm(units = 200, input_shape = dim(train_X)[2:3])  
model %>%  
  layer_dense(units = dim(test_y)[2])
```

```
model %>% compile(loss = 'mse',  
  optimizer = 'adam',  
  metrics = 'mse')
```

```
history <- model %>% fit(  
  x = train_X,  
  y = train_y,  
  batch_size = 16,  
  epochs = 50,  
  validation_split = 0.1,  
  shuffle = FALSE  
)
```

**layer\_simple\_rnn()** for RNN method

**Number of hidden units** is 200 (size of the layer)

**Input\_shape**: time steps, number of features

**Loss function** for the optimization

**Type of optimizer** and **metrics** for assessing the model performance

**Batch size**: how many samples to pass to the model at one time

**Epochs**: how many times to look at the whole dataset

**Validation\_split**: how much data to hold out for validation

```
input <- layer_input(shape = dim(train_X)[2:3])
output <- input |>
layer_lstm(units = 200) |>
layer_dense(units = 1)

model <- keras_model(inputs = input, outputs = output)
```

```
model$compile(
  loss = "mse",
  optimizer = optimizer_adam(),
  metrics = "mse"
)
```

```
history <- model$fit(
  x = train_X,
  y = train_y,
  batch_size = r_to_py(16L),
  epochs = r_to_py(50L),
  validation_split = 0.1,
  shuffle = FALSE
)
```

We set shuffle = FALSE because this is time-series data and the temporal order should be preserved during training.